Understanding Dynamic Stochastic Systems:

- Dynamic, stochastic systems involve time and contain random elements that change in a discrete manner.
    1. A **system** is a collection of entities like people and machines that interact over time to achieve one or more goals.
    2. A **model** is an abstract representation of a system, often containing structural, logical, or mathematical relationships that describe the system in terms of its state, entities, attributes, sets, processes, events, activities, and delays.
    3. The **system state** comprises variables containing all necessary information to describe the system at any given time.
    4. An **entity** is any object or component within the system that requires explicit representation in the model, such as a server, a customer, or a machine.
    5. **Attributes** are the properties of a given entity, like the priority of a waiting customer or the routing of a job through a factory.
    6. A **list** is a collection of associated entities, ordered in a logical fashion, such as customers in a waiting line ordered by arrival time or priority.
    7. An **event** is an instantaneous occurrence that changes the system's state, like the arrival of a new customer.
    8. An **event notice** records an event to occur at the current or future time, along with any necessary data to execute the event.
    9. The **event list** contains event notices for future events, ordered by their occurrence time, also known as the future event list (FEL).
    10. An **activity** is a duration of time with a specified length, like a service time or interarrival time.
    11. A **delay** is a duration of time of unspecified length, determined by system conditions, like a customer's wait time in a queue.
    12. The **CLOCK** variable represents simulated time, crucial for tracking the progression of time in the system.

Activities and Delays

- An *activity* represents a specific task or process that occurs in the system. Whether it's a service time, interarrival time, or any other defined duration, the length of an activity is known from the moment it begins.
- The duration of an activity is not affected by other events unless the model includes logic to alter or delay it.
- To keep track of activities and when they're expected to finish, we create an event notice at the start of the activity. This notice records the type of event (e.g., end-of-inspection) and the time when the activity will be completed.
- Unlike activities, the duration of a *delay* is not predetermined by the modeller but instead depends on system conditions.
- Delays often occur when certain conditions need to be met before an entity can proceed with its next action.
- For example, a customer's wait time in a queue might be influenced by factors like the number of customers ahead in line and the availability of service.
- Delays are sometimes referred to as *conditional waits*, while activities are considered *unconditional waits*.
- The completion of an activity triggers an event, often referred to as a *primary event*. This event is recorded by placing an event notice on the *future event list* (FEL).
- In contrast, delays are managed differently. When an entity experiences a delay, it's placed on another list, often representing a waiting line, until the conditions for processing are met.
- The completion of a delay, sometimes known as a *conditional* or *secondary event*, doesn't generate event notices and doesn't appear on the FEL.

The Event Scheduling/Time-Advance Algorithm

Future Event List (FEL)

- The FEL is the backbone of chronological order in simulation. It contains event notices for all future events scheduled to occur.
- When an activity begins, its completion time is calculated, and an event notice for its end is placed on the FEL.
- In real-world scenarios, events often occur randomly, but in simulations, these random events are represented by the end of certain activities, which are determined by statistical distributions.

Advancing Simulation Time

- At any given time *t*, the FEL holds scheduled future events together with their event times, ordered by their occurrence times.
- Such that the event times satisfy

$$t < t_1 \leq t_2 \leq t_3$$

- The imminent event, the next event to occur, is determined by the earliest time in the FEL. In this case t_1 is the imminent event
- When the current time (CLOCK) reaches the time of imminent event t_1:
    - The imminent event notice is removed from the FEL
    - The imminent event is executed, and a new system snapshot is created based on the old system snapshot nature of the event.
- New events may be generated during this process and are scheduled by placing event notices on the FEL.
- This cycle continues until the simulation concludes.

CLOCK	System state	...	Future event list	...
t	(5, 1, 6)		$(3, t_1)$ – Type 3 event to occur at time t_1 $(1, t_2)$ – Type 1 event to occur at time t_2 $(1, t_3)$ – Type 1 event to occur at time t_3 . . . . $(2, t_n)$ – Type 2 event to occur at time t_n	

1. Step 1: Remove Imminent Event Notice
    - Identify and remove the event notice for the imminent event from the Future Event List (FEL).
    - The imminent event is the next event scheduled to occur based on its time (t1) in the FEL.

2. Step 2: Advance Simulation Time
    - Move the simulation clock (CLOCK) forward to the time of the imminent event (t1).
    - This advancement ensures that the simulation progresses in chronological order.

3. Step 3: Execute Imminent Event
    - Execute the imminent event, which involves updating the system state, modifying entity attributes, and adjusting set memberships as necessary.
    - This step reflects the real-world impact of the event on the simulated system.

4. Step 4: Generate Future Events
    - Determine if any new events need to be generated as a result of the executed event.

- If necessary, generate these future events and place their event notices on the FEL, ensuring they are ordered by their respective event times.

5. Step 5: Update Statistics and Counters

- Update cumulative statistics and counters to reflect the changes caused by the executed event.

CLOCK	System state	...	Future event list	...
t_1	(5, 1, 5)		$(1, t_2)$ – Type 1 event to occur at time t_2 $(4, t^*)$ – Type 4 event to occur at time t^* $(1, t_3)$ – Type 1 event to occur at time t_3 . . . . $(2, t_n)$ – Type 2 event to occur at time t_n	

Efficient List Processing

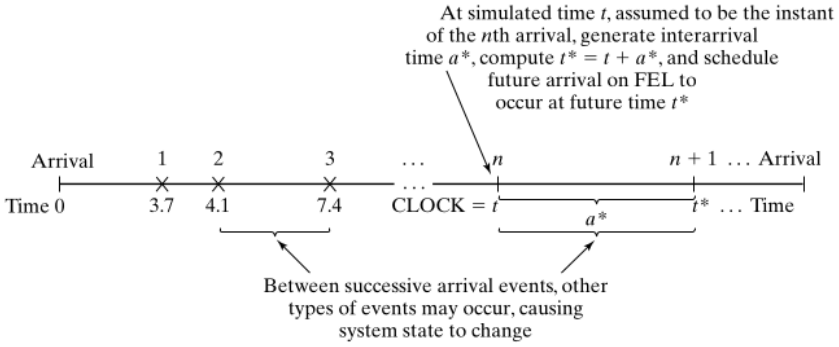
- As the simulation progresses, the FEL's length and contents change dynamically, affecting computational efficiency.
- Management of the FEL therefore becomes an important endeavour
- Operations such as removing the imminent event, adding new events, and occasionally cancelling events require efficient list-processing techniques.
- Proper logical organisation and search methods optimise list processing.

System Snapshot and Exogenous Events

- The system snapshot at time 0 is defined by initial conditions and exogenous events.
- Initial conditions establish the system state at time 0, while exogenous events are external occurrences that influence the system.
- An example of an exogenous event is the arrival of a customer in a queueing system, which is scheduled on the FEL at time 0.

Generating FutureEvents: Examples

1. Interarrival Time and Arrival Events
    - Interarrival time: This represents the duration between successive arrivals, such as customers entering a system.
    - *Bootstrapping*: To generate arrival events, a generated interarrival time (say a∗) is added to the current time (CLOCK = t), resulting in a future event time (t + a∗ = t ∗) for the next arrival.
    - Example: If interarrival times are 3.7, 0.4, and 3.3 time units, these times dictate when the next arrivals will occur.



2. Service Completion Events in Queueing Systems
    - Service completion: When one customer finishes service, a new service time (s∗) is generated for the next customer.
    - The next service completion event is scheduled at time t∗ = t + s∗, ensuring the orderly progression of service.
    - Additionally, service-completion events are generated upon customer arrival if an idle server is available.

3. Runtimes and Downtimes for Machines
    - Runtimes and downtimes: For machines subject to breakdowns, runtimes and downtimes alternate.
    - End-of-runtime events trigger downtime generation, and end-of-downtime events trigger runtime generation.
    - This alternation ensures continuous operation with periodic maintenance intervals.

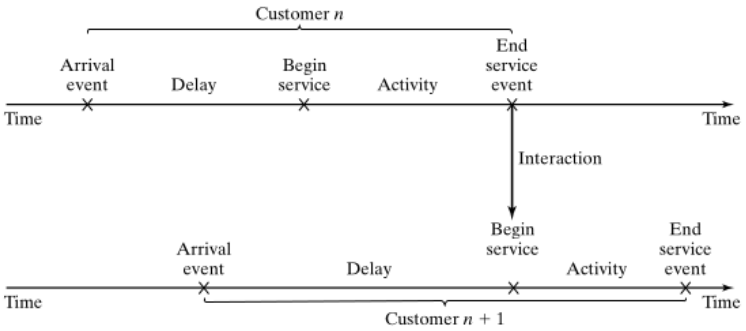
Stopping Criteria for Simulations

1. Specified End Time: Schedule a stop simulation event at a predetermined future time (T_E), defining the simulation duration.
    - Example: Simulate a job shop for 40 hours by scheduling the stop event at T_E = 40 hours.
2. Event-Driven End Time: Determine T_E based on specific events occurring during the simulation.
    - TE could be the time of the 100th service completion, a system breakdown, or the completion of a distribution centre's daily orders.
    - T_E may be unknown before the simulation and could be a statistic of interest produced by the simulation.

World Views

1. Event-Scheduling Approach
    - Focuses on events and their impact on system state.
    - Involves scheduling future events on a list for chronological execution.
    - Ideal for systems where changes occur primarily in response to discrete events.
    - Example: Arrival and departure events in a queueing system.

2. Process-Interaction Approach
    - Centres on the life cycles of entities within the system.
    - Describes interactions between entities, resources, and queues.
    - Utilises high-level constructs to represent process flows.
    - Example: Customers waiting in a queue for service.



3. Activity-Scanning Approach:
    - Emphasises activities and their triggering conditions.
    - Checks conditions at each time increment to initiate activities.
    - Provides modular models for easy maintenance and modification.
    - Example: Initiating maintenance based on machine conditions.
    - Three-Phase Approach:
        - Integrates features of event scheduling and activity scanning.
        - Divides activities into bound-to-occur (B) and conditional (C) categories.
        - Proceeds through iterative phases to optimise resource allocation and activity execution.
    - Advantages:
        - Flexibility: Offers diverse approaches tailored to system complexity.
        - Efficiency: Three-phase approach enhances runtime efficiency.
        - Resource Management: Effectively handles complex resource allocation scenarios.

References

Banks, Carson II, J., Nelson, J., Nicol, B. (2014). General Principles. In *Discrete-Event System Simulation* (pp. 90-98). Pearson.